

Implementation Plan

Take Your Trip (tytluxe.in) — Phased Build Roadmap

Version: 1.0 (Draft) **Owner:** [Your name], Full-Stack Engineer / Project Manager

Companion docs: PRD, TRD, App Flow Document, UI/UX Design Brief, Backend Schema Document

Purpose: Sequenced, deliverable-based plan to take the project from current static site to a live, working booking platform. Each phase has entry criteria, tasks, concrete deliverables, and an exit checklist — no phase should start until the prior phase's exit checklist is met.

Phase 0 — Pre-Build Decisions (Blocking — must close before Phase 1 starts)

Every prior document flagged open questions. Building without answering these risks rework. This phase is pure decision-making, not code.

Tasks:

- Resolve PRD/TRD scope conflict: confirm the phased rollout (Hotels live via TripJack+Razorpay first; Flights/Cruises/Staycations/Packages enquiry-only at launch) — TRD §0.
- Extract real brand colors from the existing logo asset to finalize the Design Brief's color tokens (§2, §13).
- Decide agent visibility model: assigned-only vs. shared queue for enquiries/bookings — Schema §14.2/§15.3.
- Decide passport-number visibility (admin-only vs. agent-visible) — Schema §15.5.
- Decide catalog CRUD permissions: can agents create/delete listings, or admin-only — Schema §14.3.
- Confirm cancellation policy terms and refund timeframes per vertical — App Flow §17.
- Confirm whether guest bookings/enquiries auto-match to an account on later registration — App Flow §17.
- Confirm TripJack contract/sandbox credentials are available, and get their actual API docs for multi-city flight and passenger data constraints — App Flow §17, Schema §9.1 note.
- Confirm Razorpay business account is live/verified (KYC can take days — start this early, it's a common launch blocker).
- Confirm Google Maps API billing account is set up (API key generation needs billing enabled even within free tier).
- Confirm data retention/PII policy with legal input — Schema §15.6.

Deliverables:

- A single "Decisions Log" document (or a locked addendum to this plan) recording every answer above, dated and owner-attributed.
- Live/sandbox credentials obtained for TripJack, Razorpay, Google Maps, and stored securely (password manager or secrets vault — not shared over chat/email).

Exit criteria: All open questions from the four prior documents have a written answer. No ambiguity should carry into Phase 1.

Phase 1 — Project Setup & Environment

Goal: A working, empty Laravel app that boots in local, staging, and is deployable — before any feature code is written.

Tasks:

- Initialize Laravel 12 project, PHP 8.2, install Filament 3, configure `.env.example` with all required keys (DB, Redis, TripJack, Razorpay, Google Maps, mail).
- Set up Git repository, branching strategy (`main` = production, `develop` = integration, feature branches), and a PR template.
- Configure Vite for Bootstrap 5 + vanilla JS asset pipeline; import Design Brief's SCSS token file (colors/typography/spacing from Design Brief §13) as the base theme override.
- Set up local dev environment (Sail/Herd) and a staging server matching TRD §7.1 (Nginx, PHP-FPM, MySQL 8, Redis).
- Configure CI pipeline (GitHub Actions or equivalent): lint (Pint), static analysis (Larastan), test runner (Pest/PHPUnit) on every PR — per TRD §7.3.
- Set up error tracking (Sentry or equivalent) wired into both local/staging and later production.
- Provision managed MySQL and S3-compatible storage (or equivalents) per TRD §7.2, even if only staging-tier for now.

Deliverables:

- Empty Laravel app running locally and on staging, deployable via the CI/CD pipeline.
- `.env.example` fully documented.
- Base Bootstrap theme reflecting brand tokens (even with no real pages yet — verify color/typography rendering on a placeholder page).
- README with setup instructions for any future developer.

Exit criteria: A blank "Hello World" Blade page renders on staging with the correct brand fonts/colors, deployed via the CI pipeline, not manually.

Phase 2 — Database & Core Models

Goal: The full schema from the Backend Schema Document exists as migrations, with Eloquent models, relationships, and factories — before any UI consumes it.

Tasks:

- Write migrations for every table in the Schema Document (§2-§11), in dependency order (destinations → hotels → hotel_images/room_types/amenities, etc.).
- Build Eloquent models with defined relationships (`hasMany`, `belongsToMany`) matching the ERD in Schema §1.
- Implement the "polymorphic-lite" resolver logic (Schema §12.1) as a trait or service class (e.g., `ResolvesVerticalReference`) used by `Enquiry`, `Offer`, `Review` models — with unit tests confirming it correctly resolves each vertical.
- Add model factories + seeders for all catalog tables (sample hotels, cruises, staycations, packages, offers) to support development/testing without live TripJack data.
- Implement soft-delete behavior where specified (Schema §12.2) and verify cascade/restrict/set-null FK behaviors match the schema exactly.
- Set up `admin_activity_logs` writing via model observers (Schema §11.2) — build this now, not bolted on later, so no admin action ships without an audit trail.

Deliverables:

- Full migration set, runnable via `php artisan migrate:fresh --seed` producing a realistic sample dataset.
- Eloquent models + relationships + factories, covered by unit tests for the polymorphic-lite resolver and FK cascade behaviors.
- ERD diagram exported (e.g., via `laravel-erd` or manually) checked into the repo for future reference.

Exit criteria: `migrate:fresh --seed` runs clean; a test suite confirms relationships resolve correctly; no schema decisions from Phase 0 are unresolved in the actual migrations.

Phase 3 — Authentication & Access Control

Goal: Customer, agent, and admin auth fully working with correct guards, roles, and policies — before building any protected UI.

Tasks:

- Install/configure Laravel Breeze (Blade) for customer auth: register, login, logout, forgot/reset password (App Flow §12).

- Implement guest-checkout support at the session/controller level (no forced login) per Schema §13.1.
- Configure Filament panel with the `admin` guard scoped to `role IN ('agent','admin') AND is_active = true` (Schema §13.2).
- Implement 2FA for admin (required) and agent (configurable) roles via Filament's 2FA or Fortify.
- Write Laravel Policy classes for every model per the permission matrix in Schema §14.2, and wire them into Filament resources.
- Implement row-level ownership scoping (Schema §15): global scopes or explicit query constraints so customers only ever see their own bookings/enquiries, agents see their configured scope (per the Phase 0 decision), and guests access confirmations only via signed URLs.
- Rate-limit login attempts (App Flow §12.2) and enquiry submissions (App Flow §11) per TRD §8.

Deliverables:

- Working customer registration/login/logout/password-reset flows on staging.
- Working `/admin` login restricted correctly by role, with 2FA enforced for admin.
- Policy test suite: automated tests proving a customer cannot access another customer's booking (expects 404), an agent cannot delete a listing (if Phase 0 decided admin-only), etc. — one test per row in Schema §14.2's matrix at minimum.

Exit criteria: Every access-control rule from the Schema Document's §14/§15 has a passing automated test, not just manual verification.

Phase 4 — Core UI: Layout, Design System & Static Pages

Goal: The global shell (header, footer, nav, toasts, buttons, cards, forms) and content-only pages are built and match the Design Brief exactly — this is the foundation every later feature page reuses.

Tasks:

- Build the global Blade layout: header/nav (§App Flow §2.1), footer (§2.2), sticky WhatsApp button, toast system, loading/skeleton components (§2.4-2.5) — as reusable Blade components.
- Implement the full component library from the Design Brief §5: buttons (primary/accent/secondary/tertiary/disabled/loading states), cards, form inputs with validation states, badges, modals — as a Blade component library (`<x-button>`), `<x-card>`, `<x-badge status="confirmed">`, etc.) so every later page composes from the same primitives instead of redefining styles.

- Build Home (§App Flow §3), About, Contact (§4), and static error pages (404/500/maintenance, §14) — these have no complex data dependencies and validate the design system end-to-end early.
- Implement responsive behavior per Design Brief §8 (mobile-first breakpoints, sticky bottom bars, filter drawers as reusable patterns).
- Implement newsletter subscribe (§2.2) and Contact form (§4.2) against the `enquiries`/`newsletter_subscribers` tables.

Deliverables:

- A Blade component library documented (e.g., a `/dev/components` style-guide route showing every component/state) for the rest of the build to reuse.
- Home, About, Contact, and error pages live on staging, fully responsive, matching the Design Brief.
- Contact form and newsletter signup functionally working end-to-end (data lands in DB, confirmation states render correctly).

Exit criteria: A design/stakeholder review confirms these pages match the Design Brief's tone and component rules; the component library is reused (not duplicated) starting in Phase 5.

Phase 5 — Core Features: Catalog Browsing (Cruises, Staycations, Packages)

Goal: Ship the simpler, curated (non-TripJack) verticals first — they validate the enquiry flow and admin CMS without third-party API risk, and de-risk the pattern before tackling Hotels/Flights.

Tasks:

- Build Cruises, Staycations, Packages listing pages (filter chips, grid, empty/error states) per App Flow §7–§9.
- Build their respective detail pages (image gallery, itinerary/amenities/inclusions, Enquire Now CTA).
- Build the shared Enquiry Form component (App Flow §11) with full validation, success/error states, and WhatsApp contextual fallback — this component is reused by Hotels/Flights in Phase 6, so build it once, correctly, here.
- Build the corresponding Filament admin resources: Cruises, Staycations, Packages, and the Enquiries resource with status pipeline + agent assignment (App Flow §15.2, §15.4).
- Build the Offers module: public Offers page (§10) + Filament Offers resource (§15.3) + the scheduled job that expires offers past `valid_to` (TRD §3.4/§7.3).

Deliverables:

- Cruises/Staycations/Packages fully browsable and enquire-able on staging, with real (seeded/admin-entered) content.
- Admin can create/edit/deactivate listings and offers without developer involvement — validate this literally by having a non-developer stakeholder try it.
- Enquiries submitted on the public site appear correctly in the Filament Enquiries resource with notifications firing.

Exit criteria: An end-to-end enquiry (public form submit → DB record → admin notification → status update in Filament) works with zero manual DB intervention.

Phase 6 — Core Features: Hotels & Flights (Search, Browse, Detail)

Goal: Build the TripJack-dependent search/browse experience, still stopping short of booking/payment — isolates integration risk from transaction risk.

Tasks:

- Build the TripJack API client (`Support/TripJack`), TRD §1.3) with DTOs/response mappers for hotel and flight search, using sandbox credentials.
- Build Hotel listing/search (§App Flow §5.1–5.2) merging live TripJack results with curated/manual `hotels` table rows (Schema §3.1's `source` field), including skeleton loading and the TripJack-failure fallback state.
- Build Hotel detail page (§5.3) with static content from DB + live room/pricing from TripJack, including the "Enquire Instead" fallback.
- Build Flight search/results (§6.1–6.2), including the no-catalog-fallback error state since flights aren't persisted (Schema §9 note).
- Build destination/airport autocomplete backed by cached TripJack static data (TRD §6.1's nightly sync job).
- Build the Filament Hotels resource with geocoding via Google Maps on save (App Flow §15.1).

Deliverables:

- Hotel and Flight search/browse fully functional against TripJack sandbox on staging, including all defined empty/error/loading states.
- Google Maps embed working correctly on hotel detail pages.
- A documented fallback behavior test: simulate TripJack timeout/failure and confirm the UI shows the specified fallback (curated listings or error+WhatsApp CTA), not a blank/broken page.

Exit criteria: A QA pass confirms every state defined in App Flow §5–§6 (loading, empty, error, success) renders correctly, including deliberately forced API failures.

Phase 7 — Booking & Payment Integration

Goal: The highest-risk phase — real money, real inventory holds. Built last among features, tested hardest.

Tasks:

- Build the Hotel booking flow (App Flow §5.4): Room & Dates confirm → Guest Details → Payment → Confirmation, including the TripJack hold/confirm sequence and all specified error states (room no longer available, hold failure, price change).
- Build the Flight booking flow (§6.3): Fare Review (with re-pricing check) → Traveler Details (multi-passenger) → Payment → Confirmation.
- Integrate Razorpay: server-side order creation, Checkout.js client integration, and — critically — the **server-side webhook handler** as the sole source of truth for payment status (TRD §6.2, App Flow §16 rule #7). Build the polling mechanism for the client-side "Confirming your booking..." state.
- Implement the `failed_needs_review` booking status path end-to-end: payment succeeds but TripJack confirm fails → booking flagged, admin alerted (TRD §6.1, App Flow §5.4 Step 3, Schema §9.1) — this specific failure path must be deliberately tested, not just hoped to never happen.
- Build the customer Account area: Overview, My Bookings (with Cancel Booking → TripJack cancel + Razorpay refund), My Enquiries, Profile (App Flow §13).
- Build the admin Bookings and Payments resources, including manual status override (audit-logged) and refund trigger (App Flow §15.5–§15.6).
- Implement booking confirmation emails with PDF voucher/invoice attachment.

Deliverables:

- Full hotel booking, in sandbox, from search through paid confirmation, including email delivery.
- Full flight booking, in sandbox, same scope.
- A written test log covering every failure state explicitly listed in App Flow §5.4/§6.3: payment cancelled mid-flow, payment declined, TripJack confirm failure post-payment, polling timeout, fare re-pricing — each with confirmed correct UI behavior.
- Admin can view, override, and refund a booking end-to-end in Filament.

Exit criteria: No booking can reach `confirmed` status without a webhook-verified payment (verify this by attempting to spoof the client-side success callback in a test and confirming it does **not** confirm the booking). This is a hard security/financial gate — do not proceed to Phase 8 until proven.

Phase 8 — Testing (Cross-Cutting Hardening Pass)

Goal: A dedicated phase to test the whole system together, not just per-feature — real gaps show up at integration seams.

Tasks:

- **Functional QA:** manually walk every screen/state in the App Flow Document against the live staging build — treat it as the literal test script/checklist, screen by screen.
- **Automated test coverage:** ensure Policy tests (Phase 3), model relationship tests (Phase 2), and critical booking/payment path tests (Phase 7) all run in CI; add missing coverage for any gap found during QA.
- **Security pass:** verify TRD §8 checklist item by item — CSRF, XSS escaping audit (especially any `{!! !!}` Blade output from admin rich-text fields), rate limiting, webhook signature verification, PII/passport encryption, admin 2FA enforcement, dependency audit (`composer audit`).
- **Load/performance check:** basic load test on search endpoints (Hotels/Flights) since these hit external TripJack APIs and could bottleneck under concurrent users; verify caching (TRD §6.1) is actually reducing redundant TripJack calls.
- **Cross-device/browser QA:** test on real mobile devices (not just responsive dev-tools resize) per Design Brief §8 — sticky bottom bars, filter drawers, tap targets.
- **Accessibility audit:** run an automated tool (axe, Lighthouse) plus manual keyboard-navigation pass against Design Brief §11's WCAG AA baseline.
- **Payment edge-case testing:** use Razorpay's test card/failure simulation tools to trigger every failure mode listed in Phase 7, plus double-submission/double-click protection checks.
- **Content review:** verify all copy (error messages, button labels, empty states) has passed the copywriting review flagged in App Flow §17 — placeholder/draft copy should not ship to production.

Deliverables:

- A QA sign-off checklist mapped directly to App Flow Document sections, each item marked pass/fail with evidence (screenshot or test log).
- A security checklist mapped to TRD §8, signed off.
- A bug tracker with all findings triaged (blocker/major/minor) and blockers resolved before Phase 9.

Exit criteria: Zero open blocker-severity bugs; all automated tests green in CI; security and accessibility checklists fully signed off.

Phase 9 — Deployment (Production Launch)

Goal: Go live safely, with a defined rollback path.

Tasks:

- Provision production infrastructure per TRD §7.2 (app server, managed MySQL, Redis, S3-equivalent storage, CDN).
- Switch all third-party credentials from sandbox to live (TripJack, Razorpay, Google Maps) — verify each individually in a production-like environment before DNS cutover, per TRD §7.4.
- Recreate/migrate static content (About, testimonials, brand assets) from the current WordPress site into seeded production data (TRD §7.4 — rebuild, not DB migration).
- Set up production monitoring (Sentry, uptime monitoring, queue worker monitoring via Supervisor) and confirm alerting actually reaches someone (test the `failed_needs_review` alert path in production-like conditions one more time).
- Configure automated daily DB backups with a tested restore procedure (TRD §8) — actually perform one test restore, don't just assume the backup job works.
- Prepare and rehearse a maintenance-mode cutover window; keep the maintenance page (App Flow §14.3) ready.
- DNS cutover for tytluxe.in to the new application, with a rollback plan (keep the old static site deployable/reachable at a fallback path for a short window in case of critical issues).
- Post-launch smoke test: complete one real, small, end-to-end hotel booking with real payment (refunded immediately after) to confirm the live integration genuinely works, not just in sandbox.

Deliverables:

- Live production site at tytluxe.in.
- A verified backup/restore procedure.
- A documented rollback plan, tested or at least dry-run reviewed with the team.
- A completed real-money smoke-test booking, refunded, with a written confirmation it worked end-to-end.

Exit criteria: Site is live, monitored, backed up, and one real transaction has been verified successful in production.

Phase 10 — Final Polish & Post-Launch Stabilization

Goal: The first 2–4 weeks post-launch, converting real user behavior into fixes and refinements rather than shipping new scope.

Tasks:

- Monitor error tracking and the `failed_needs_review` queue daily; triage and fix any real-world edge cases TripJack/Razorpay expose that sandbox testing didn't catch.
- Review the PRD's success metrics (§7): enquiry volume, WhatsApp CTR, form completion rate, mobile vs. desktop split — confirm analytics are actually capturing these correctly, fix tracking gaps found.
- Content polish pass: replace any remaining placeholder copy, verify all images are optimized/responsive (Design Brief §8), confirm brand color/typography consistency site-wide against the Design Brief.
- Performance tuning based on real traffic: query optimization for any slow admin/report queries, cache tuning for TripJack search results.
- Collect and action first-week feedback from actual agents/admins using Filament day-to-day — the PRD's "admin can manage listings without a developer" promise should be validated by real usage, not just a walkthrough.
- Freeze scope creep explicitly: log any new feature requests (e.g., items from PRD §8's "avoid in v1" list) into a v2 backlog rather than patching them into the stabilizing v1 build.

Deliverables:

- A post-launch stabilization report: bugs found/fixed, metrics baseline established against PRD §7 targets.
- A v2 backlog document capturing deferred features (from PRD §8, TRD phasing, App Flow §17) as prioritized future work, not ad hoc scope creep into v1.

Exit criteria: Error rates and booking failure rates stable at an acceptable baseline for two consecutive weeks; success metrics are being measured correctly; the team formally hands off from "launch mode" to "steady-state operations."

Summary Timeline View

Phase	Focus	Key Risk if Skipped/Rushed
0	Decisions	Rework from unresolved ambiguity
1	Setup	Broken foundation, inconsistent environments
2	Database	Schema drift, data integrity bugs later
3	Auth & Access	Security/privacy breach (wrong user sees wrong data)
4	Core UI/Design System	Inconsistent UI, duplicated component code
5	Curated Verticals	No validated enquiry pipeline before tackling harder verticals
6	Hotels/Flights Search	TripJack integration risk hits page after payment risk if reordered
7	Booking & Payment	Highest risk phase — financial/inventory correctness
8	Testing	Bugs found by customers instead of QA
9	Deployment	Failed/rocky launch, no rollback plan
10	Polish & Stabilization	Real issues ignored in favor of chasing new features

Sequencing rationale: Curated verticals (Phase 5) intentionally come before Hotels/Flights (Phase 6), and search/browse (Phase 6) comes before booking/payment (Phase 7) — each phase isolates one new category of risk (content/CMS risk → third-party API risk → financial transaction risk) rather than combining all of it into one "build everything" phase, so failures are caught early and cheaply rather than late and expensively.

This plan assumes Phase 0's decisions are genuinely closed before Phase 1 begins. If any decision is deferred "for now," track it explicitly as a risk in the phase it affects rather than letting it silently resurface during build.